ParkVision

Parking Lot Occupancy Detection

Yusuf Shahzad

Course: ITAI 1378 – Computer Vision & AI

Tier: Tier 2 — Object Detection

The Problem

30%

of urban traffic is circling for parking

7.8 min

average time wasted searching per trip

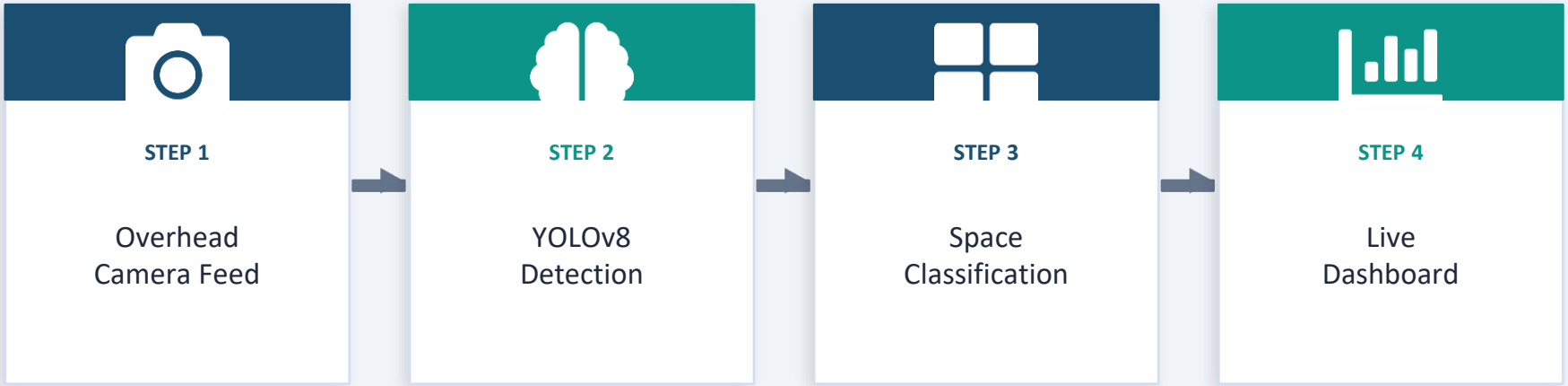
$97B

in fuel & productivity lost annually (USA)

Drivers and parking operators have no real-time visibility into which spots are available — causing wasted time, increased emissions, and frustrated drivers.

Our Solution

A real-time system that uses overhead camera feeds and YOLOv8 to detect and count occupied vs. available parking spaces.



Who Benefits: Parking lot operators, shopping centers, airports, university campuses

Technical Approach

CV TECHNIQUE

Object Detection

Detects individual cars in overhead images and classifies each space as empty or occupied.

MODEL

YOLOv8n / YOLOv8s

Ultralytics YOLOv8 — fast, accurate, and pre-trained on COCO including 'car' class. Nano or small variant for real-time speed.

FRAMEWORK

PyTorch + Ultralytics

Ultralytics provides a clean API around YOLOv8 for training, fine-tuning, and inference. Runs on Colab GPU.

Data Plan

Primary Dataset

PKLot Dataset
~12,000 images from
3 Brazilian parking lots
(sunny, cloudy, rainy)

Backup Source

Roboflow Universe
'Parking Lot Detection'
public dataset ~5,000
annotated images

Labels

Two classes:
- occupied (car present)
- empty (space free)

Bounding box annotations

Preprocessing

- Resize to 640×640
- Normalize pixels
- Augment: flip, rotate, brightness jitter

Total: ~12,000 labeled images | Train/Val/Test: 70% / 15% / 15%

System Diagram

Success Metrics

≥ 90%

Detection Accuracy

≥ 85%

mAP@0.5 Score

< 1s

Per-Frame Latency

Metric	What it Measures	Target	Priority
Accuracy	% of spaces correctly classified	≥ 90%	🥇 Primary
mAP@0.5	Mean average precision at IoU 0.5	≥ 85%	🥇 Primary
Inference Speed	Time to process one frame	< 1 second	🥈 Secondary
False Negatives	Occupied spaces missed	< 5%	🥈 Secondary

Week-by-Week Plan

Wk 10	Download PKLot dataset, set up GitHub repo, configure Colab environment	✅ Dataset ready
Wk 11	Train YOLOv8 baseline on PKLot. Run first evaluation. Tune hyperparameters.	✅ Model running
Wk 12	Add data augmentation, fine-tune model, reach ≥90% accuracy target	✅ Target accuracy
Wk 13	Build visual overlay (green/red space mapping). Create demo video.	✅ Demo ready
Wk 14	Write documentation, clean up code, finalize README and report	✅ Fully documented
Wk 15	Final polish and rehearsal	🎉 Present!

Challenges & Backup Plans

Low accuracy on occluded cars

Plan B: Use data augmentation (partial occlusion). Try larger model (YOLOv8m).

Med Risk

PKLot dataset doesn't match target lot

Plan B: Supplement with Roboflow parking dataset or collect 100+ custom images.

Med Risk

Model too slow for real-time on CPU

Plan B: Switch to YOLOv8n (nano) or use static images instead of live feed.

Low Risk

Overfitting / poor generalization

Plan B: Add dropout, reduce model size, add more augmentation, use early stopping.

Med Risk

Colab GPU quota runs out

Plan B: Switch to Kaggle (30hr/week free GPU) or use HEIDI HPC cluster.

Low Risk

Resources Needed

Compute — $0

- Google Colab (free T4 GPU)
- Kaggle Notebooks (30h/wk GPU)
- HEIDI HPC (campus cluster)

Frameworks — $0

- PyTorch 2.x
- Ultralytics YOLOv8
- OpenCV, Matplotlib

Dataset — $0–$5

- PKLot (free, public)
- Roboflow Universe (free)
- Optional: Roboflow annotate

Tools — $0

- GitHub (free)
- Jupyter Notebooks
- LabelImg (if labeling needed)

💰 **Estimated Total Cost: $0 – $5 | All tools and datasets are free or open-source**